

Algorithm–Schedule Separation for VLM Image Preprocessing

Julien Eric Darve
jdarve@stanford.edu

June 2026

1 Background and Setup

Vision-language models (VLMs) must transform input images into tensors before the model can process them. Historically, computer vision models accepted a single fixed image resolution, so preprocessing was often limited to resizing, rescaling, and normalization. Modern VLMs instead preserve more of an image’s native resolution and use model-specific operations such as dynamic resizing, tiling, padding, patchification, and layout conversion. These pipelines can process millions of pixels per image and may execute entirely on the CPU before a GPU can begin model inference. As GPU inference becomes faster, inefficient CPU preprocessing can become a meaningful yet easily overlooked end-to-end bottleneck.

The input to the system is a batch of decoded RGB PIL images with potentially different dimensions and aspect ratios. The output is the exact floating-point tensor representation expected by an existing VLM. For Qwen2.5-VL, this is a matrix of flattened image patches together with metadata describing the temporal, height, and width patch grid. For LLaVA-NeXT, this is a set of channel-first 336×336 image tiles selected by its AnyRes policy. The output shape is data-dependent: larger images, different aspect ratios, and different model policies can all change the amount of output produced.

Existing open-source processors express these pipelines as a sequence of library operations. A stage may produce a complete intermediate image or tensor that is consumed by the next stage, even when the consumer performs only a pointwise operation or a layout transformation. Implementations may also convert between PIL images, NumPy arrays, and PyTorch tensors. These choices are simple and modular, but they can increase runtime, memory traffic, and peak memory usage without changing the mathematical result.

Our goal is to improve VLM image preprocessing by changing its *schedule*, not its algorithm. A valid solution is constrained by the fact that it must preserve the model-specific resize policy, normalization values, tiling or patchification behavior, output layout, and metadata. It must support batches of

variable-size images and remain useful across pipelines with different algorithm structures. We evaluate CPU execution because preprocessing commonly runs on the host while the accelerator is reserved for model inference. We optimize runtime and peak memory allocation, but do not change the VLM architecture, reduce input resolution, approximate the output layout, or include model inference in the measured system.

The central question of this project is:

Is VLM image preprocessing a suitable application for separating algorithm from schedule?

Our falsifiable hypothesis is that common VLM preprocessing pipelines contain enough avoidable data movement that multiple schedules for the same fixed algorithm will produce meaningfully different runtime and memory behavior. Furthermore, a shared scheduling vocabulary should apply to more than one model-specific pipeline. The hypothesis would be weakened if changing the schedule had little effect, if performance were dominated entirely by irreducible interpolation compute, or if each model required a separate one-off optimization that could not be expressed through the same DSL.

The technical crux is that eliminating intermediate buffers changes how the final output must be constructed. It is straightforward to execute `resize`, `normalize`, and `patchify` as separate passes: each stage reads a conventional dense image produced by the previous stage. However, a fully fused implementation must determine where every sampled pixel belongs in the final model-specific tensor without first materializing that image. Qwen uses a non-row-major merge-block patch ordering, while LLaVA uses dynamic AnyRes tiles and channel-first output. The difficult part is therefore to represent these distinct algorithms while allowing the scheduler to fuse their loops, parallelize independent work, and scatter values directly into a preallocated output without changing the result.

2 Approach

2.1 Starting Point: Hugging Face Image Processors

We started from the open-source Hugging Face `AutoImageProcessor` implementations for Qwen2.5-VL and LLaVA-NeXT. Hugging Face implements the image pre-processing algorithm for these two models. We exactly reproduce their algorithm. In order to implement a DSL that explores varying schedule, the implementation had to be re-written and input-output matched to Hugging Face.

2.2 Separating Algorithm from Schedule

Our DSL is inspired by Halide’s separation between *what* a program computes and *how* it is executed. Qwen is represented by the algorithm DAG

`resize → rescale → normalize → patchify.`

The `resize` stage applies Qwen’s smart-resize policy, while patchification produces Qwen’s merge-block patch ordering and temporal duplication. LLaVA uses a different DAG,

`anyres tile → rescale → normalize.`

Its `anyres tile` source operation captures AnyRes resolution selection, the thumbnail resize, the aspect-ratio-preserving grid resize, padding, and 336×336 tile extraction.

Listing 1 overviews the central abstraction: the Qwen algorithm is defined once, while only the schedule changes.

Listing 1: Pseudocode Qwen algorithm and schedules

```
pipeline = (Pipeline()
    .add(Func("resize", "resize", [], resize_params))
    .add(Func("rescale", "rescale", ["resize"], {"scale": 1.0 / 255.0}))
    .add(Func("normalize", "normalize", ["rescale"], norm_params))
    .add(Func("patchify", "patchify", ["normalize"], patch_params))
    pipeline.output = "patchify"

sched_v1 = Schedule(stages={
    "resize": StageSchedule(parallel="batch")})

sched_v2 = Schedule(stages={
    "resize": StageSchedule(parallel="batch"),
    "rescale": StageSchedule(compute_at="inline"),
    "normalize": StageSchedule(compute_at="inline")})

sched_v3 = Schedule(stages={
    "resize": StageSchedule(parallel="batch"),
    "rescale": StageSchedule(compute_at="inline"),
    "normalize": StageSchedule(compute_at="inline"),
    "patchify": StageSchedule(
        compute_at="inline", store_at="root",
        write_via="qwen_patch_coords")},
    preallocate_output=True)
```

2.3 Scheduling Primitives

The current backend intentionally implements the small set of scheduling primitives needed by the evaluated pipelines. The `compute_at` annotation controls whether a stage runs in its own loop nest or is inlined into another stage’s loop. The `store_at` annotation controls whether a result is materialized as a separate buffer or stored at the output root. The pipeline-level `preallocate_output` flag allocates the final output once before the pixel-processing kernel begins.

The `write_via` primitive is necessary because the final output is not always row-major. This is one of the key technical cruxes. Qwen patches use a merge-block ordering, so a fully fused resize loop must compute the destination patch row and column for every output pixel. LLaVA similarly writes directly into a channel-first tile tensor. The algorithm supplies a pure model-specific coordinate function, while the schedule chooses whether to invoke that function inline rather than first materializing a spatial image. Thus, in order to implement our fully fused kernel, we perform a scatter operation (loop over input pixels) instead of a gather (loop over output patches).

We evaluate three schedules as a controlled ablation:

1. **v1: staged execution.** Every operation runs in its own loop and materializes its full output buffer.
2. **v2: pointwise fusion.** Rescale and normalize are inlined into the sampling loop. Qwen retains one normalized spatial-image intermediate before patchification, while LLaVA retains one sampled tile intermediate.
3. **v3: full fusion and direct output writes.** Resize, pointwise operations, layout conversion, and output addressing are fused into one processing loop that writes into a preallocated output tensor.

The `parallel` primitive maps independent preprocessing units to Numba `prange` worker threads. For Qwen, the outer loop is over input images: each worker processes one complete image and writes to a disjoint range of the final patch tensor. For LLaVA, the backend flattens the dynamically generated tiles and parallelizes over output tiles, which provides finer-grained work units. Within each image or tile, the output-pixel, channel, and bilinear-sampling loops remain serial. We do not explicitly schedule SIMD vector lanes or parallelize individual pixels; vectorization is left to Numba.

2.4 Code Generation

The backend currently performs schedule classification and template binding rather than compiling arbitrary source code. This keeps the implementation small while ensuring that the selected kernel is derived from the schedule rather than chosen by a model-specific configuration flag. The Qwen and LLaVA `build` function validates the DAG, gathers algorithm parameters, classifies the schedule as `naive`, `pointwise`, or `full`, and returns the corresponding Numba template. Cheap dynamic control work remains in Python, such as smart-resize dimensions, prefix-sum patch offsets, and AnyRes resolution.

2.5 Implementation Process

We began our implementation with three hand-tuned Qwen kernels for the same v1, v2, and v3 experiments as explained above, but without all of the underlying DSL frameworks. Only after these kernels produced correct outputs did we

formalize their differences as schedule annotations and reproduce them through the DSL backend. We ran our benchmarks against the hand tuned kernels both as a sanity check of our DSL, and to inform which primitives we should aim to support. After that, we generalized the DSL to LLaVA-NeXT. This required adding a tile algorithm primitive with AnyRes, helping us test whether the same schedule vocabulary could support a substantially different algorithm with dynamic AnyRes tiling rather than Qwen patchification.

Before writing fused kernels, we tried to accelerate the existing Hugging Face pipelines by wrapping them in `torch.compile`. We thought this would provide a more robust baseline to Numba, which is also a compiled language. This did not work. Compilation was roughly neutral on the NumPy/PIL legacy backend and $2\text{--}3\times$ slower on the TorchVision fast backend across all three workloads. The cause is that these pipelines interleave PIL image operations with tensor operations, so the compiler cannot capture an end-to-end graph: it traces up to each PIL call, breaks, falls back to eager execution, and re-enters afterward. These graph breaks erase any fusion benefit and add dispatch overhead. The negative result showed that a compiler can only help once the whole pipeline lives in a single compilable stack, which is exactly what motivated our unified Numba backend and the fully fused v3 schedule.

3 Results

3.1 Definition of Success

We evaluate the broader question: *Is VLM image preprocessing a suitable application for separating algorithm from schedule?* We define success using three measurable criteria.

1. **The DSL improves on open-source implementations.** Each DSL schedule must produce the same model inputs as the algorithm-matched Hugging Face bilinear processor, including output values, shapes, ordering, and metadata. Given correctness, we measure whether the DSL reduces median runtime or peak RSS memory relative to Hugging Face. We do not require the DSL to win every runtime benchmark, as the Hugging Face implementations use the highly optimized TorchVision backend. However, consistent memory reduction or runtime improvement in data-movement-bound workloads would demonstrate that the open-source implementations leave performance on the table.
2. **Schedule choice changes performance.** Qwen / LLaVA v1, v2, and v3 are generated from one algorithm DAG and must produce equivalent outputs. Success requires the schedules to exhibit a measurable runtime or memory difference beyond benchmark noise. In particular, pointwise fusion should remove intermediate buffers, full fusion and direct writes should approach the final-output memory floor, and the parallel schedule should scale as additional physical CPU cores are used.

3. **The scheduling abstraction applies across model-specific algorithms.** A DSL is only justified if it provides more than a one-off optimized Qwen kernel. We therefore require the same scheduling vocabulary to express both Qwen’s smart-resize and patchification pipeline and LLaVA-NeXT’s dynamic AnyRes tiling pipeline. Success means both models can be represented and validated using the shared primitives, even if their optimal performance behavior differs.

These criteria distinguish two possible outcomes. If scheduling improves performance, the DSL identifies and removes avoidable data movement. If a correct fully fused schedule does not improve runtime, profiling should show that the remaining cost is irreducible computation, revealing a boundary of the approach rather than a failure of correctness.

3.2 Experimental Setup

Our primary goal with our benchmarking setup was to replicate a realistic model preprocessing pipeline. To that goal, we evaluated memory and runtime performance on a diverse set of workloads. We explore two unique, complex VLMs compared against algorithm-matched standard open source baselines and rigorously check correctness.

Models and Baselines. We selected Qwen2.5-VL and LLaVA-NeXT as our SOTA, open-source, native resolution VLM image preprocessing pipelines. We compared our DSL implementation to the following baselines

1. **HF Legacy:** Numpy/PIL backend.
2. **HF Fast:** TorchVision-V2/PIL backend. The TorchVision backend was Hugging Face and PyTorch’s solution for fast pre-processing, and it is a strong baseline.
3. **HF Bilinear:** The standard Qwen and LLaVA resize interpolation kernels use bicubic interpolation. Re-implementing bicubic interpolation in Numba was out of scope for this project; instead, we implemented a simpler bilinear kernel with anti-aliasing. We replace the resizing kernel in Qwen to use bilinear instead of bicubic to have an exactly algorithm-matched baseline.

Verification. We measured correctness by running the Hugging Face and our DSL implementations on the same inputs and confirming the same output shapes and values. Our kernels reproduce Hugging Face bilinear up to 1 raw pixel level, meaning up to the least significant bit for uint8. This is near optimal, and any remaining differences are due to floating point precision errors.

Workloads. We sought to explore a diverse, representative set of image workloads. Our input are standard PIL images in RGB uint8 (0-255) format. We use uniformly random pixel values, as the pre-processing pipelines are input independent. We explore three different **workloads** of input images:

1. **W2:** 32 1024×1024 images. W2 measures standard homogeneous image input.
2. **W3:** 32 images of mixed resolution, with dimensions randomly sampled from 256 to 1024. W3 explores if the pipeline is robust to non-uniform sized inputs, representing a real-life image dataset.
3. **W4:** 8 2480×3508 images. W4 is a large image stress test calibrated to be an A4 document at 300 DPI.

Evaluation Metrics. For W2 and W3, we run 10 warmup batches, then report the median of 30 batches. For W4, this is reduced to 2 warmup steps and 16 timing steps. We evaluate runtime performance by recording the milliseconds per image. To compute memory allocation, we sample Resident Set Size (RSS) at 4000hz: we report the total amount of memory allocated by the process subtracting the input memory. This is measured by the underlying Linux operating system, so it generalizes to all backends: torch, numpy, numba, python. Profiling is done for Hugging Face using cProfile, which measures the amount of time in each function. Our profiling is not able to time individual operations in our Numba Kernels as they are end to end kernels; we focus on a timing comparison between input conversion with the kernel itself.

Compute. Experiments ran on an AWS c7i.4xlarge instance with 16 vCPUs corresponding to 8 physical Intel Xeon cores, 32 GiB RAM, and Ubuntu 24.04. The machine uses 4th Gen Intel Xeon Scalable processors. These are high end machines; in practice CPU quality may vary and runtime results for pre-processing can be an even larger effect on CPU utilization for VLM pipelines.

3.3 Result 1: Storage Fusion Drives Peak Memory to the Output Floor

Our method significantly optimizes memory allocation compare to Hugging Face. Figure 1 reports single-thread peak/output RSS. Hugging Face holds $1.7\text{--}4.1\times$ the output size in live memory at the peak (HF Fast is the worst because the torchvision path keeps the most intermediates); the DSL’s fully-fused v3 reaches the output floor: $1.00\times$ on W2 and W3 and $1.08\times$ on W4 for Qwen, and $1.00\times$ on all three for LLaVA. This result aligns with our scheduling choice: v3 allocates the output once and scatters into it, so there is by construction nothing else live. It generalizes across both models and all three workloads, which is the strongest evidence for the hypothesis.

3.4 Result 2: Parallelism Schedule Choice Optimizes Runtime

Figure 2 investigates runtime on Qwen W4 between 1, 4, and 8 threads. The DSL starts *behind* HF Bilinear at one thread (994 vs. 638ms). A single serial `njit` bilinear loop has difficulty matching torchvision’s highly optimized `resize` (discussed in result 5: profiling). But the DSL’s batch-parallel schedule scales

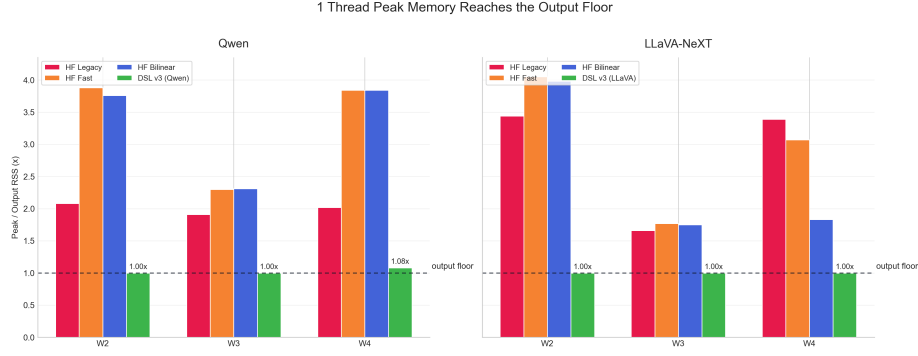


Figure 1: **Storage Fusion Drives Peak Memory to the Output Floor.** Qwen and LLaVA-NeXT on all three workloads (x-axis) measuring peak memory allocation (y-axis). Results are shown for one threaded results. We compare the three HF baselines to our fully-fused DSL implementation

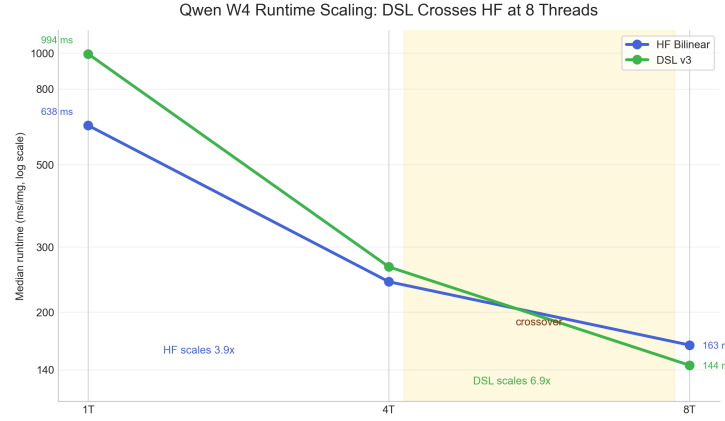


Figure 2: **Parallelism Schedule Choice Optimizes Runtime.** Qwen on W4 parallelized evaluating runtime (ms/img y-axis) between 1, 4, and 8 threads (x-axis), comparing HF to our fully-fused DSL implementation

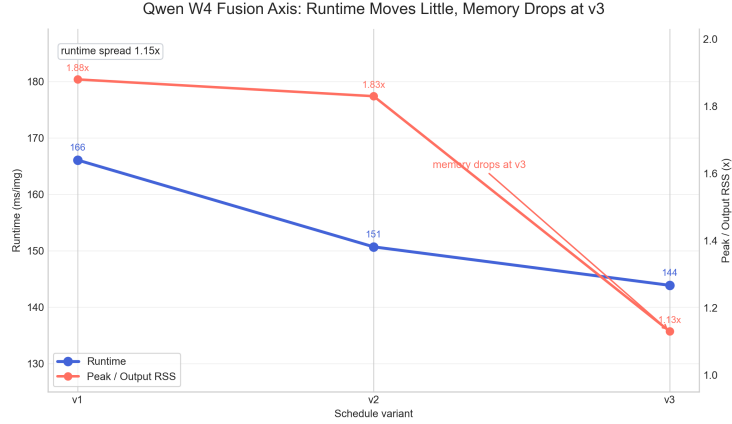


Figure 3: **Fusion, Scatter Schedule Choices Optimize Memory.** Our Qwen DSL implementation runtime (ms/img, y-axis) on W4 between the v1 (naive), v2 (pointwise fusion), v3 (full fusion) schedules (x-axis)

better ($6.9\times$ from 1 to 8 threads vs. HF’s $3.9\times$), so it crosses over and wins at 8 threads (144 vs. 163 ms). This result shows that parallelism is the schedule choice we can tune to optimize runtime.

3.5 Result 3: Fusion, Scatter Schedule Choices Optimize Memory

Figure 3 isolates the $v1 \rightarrow v2 \rightarrow v3$ schedule axis on Qwen W4 at fixed thread count. Runtime does not move significantly (spread $\approx 1.15\times$: $166 \rightarrow 151 \rightarrow 144$ ms), but peak memory collapses at v3 ($1.88\times \rightarrow 1.83\times \rightarrow 1.13\times$). While parallelism in Result 2 optimized runtime, fusion does not, and instead optimizes memory. Thus, fusion is the schedule choice we can tune to optimize memory movement.

3.6 Result 4: Where Scheduling Doesn’t Help: LLaVA is Compute-Bound

Figure 4 shows LLaVA-NeXT runtime at 8 threads normalized to HF Bilinear. The three DSL schedules are nearly identical within each workload; the fusion axis does not improve runtime significantly for LLaVA as well as Qwen. The DSL is competitive on the smaller image workloads ($1.04\times$ on W2, $0.87\times$ on W3), but $1.70\times$ slower on the large W4. LLaVA’s AnyRes path is dominated by a large interpolation resize, exactly the operation where our kernel is weakest and torchvision’s robustly optimized resize is strongest. Scheduling cannot recover this; there are no intermediate buffers to fuse and no data-movement overhead to remove. The memory result still holds for LLaVA (Result 1: $1.00\times$ on all

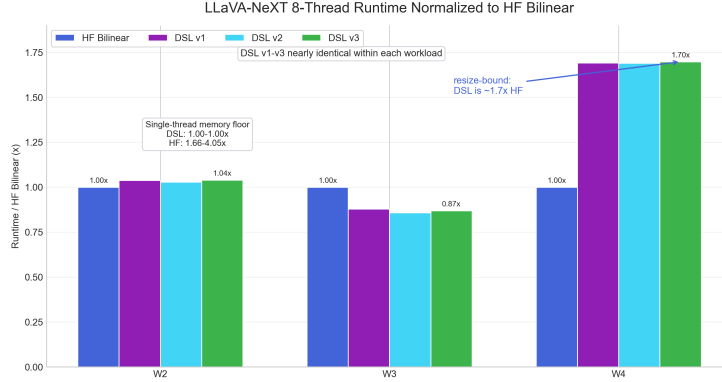


Figure 4: **Where Scheduling Doesn’t Help: LLaVA is Compute-Bound.** LLaVA-NeXT on all three workloads (x-axis) measuring runtime (ms/img, y-axis). Results are shown for the multi-threaded implementation. We compare Hf Bilinear to our v1, v2, v3 scheduling ablations.

workloads), so the abstraction’s benefit is model-dependent. The runtime only improved when the pipeline is data-movement-bound rather than interpolation-bound. We dig into this claim deeper in the following section.

3.7 Result 5: Profiling Qwen and LLaVA

To confirm our claims about runtime differences between the different models, we profile the Hugging Face implementations by category in Figure 5 on the W4 workload. The Hugging Face implementations were profiled by their individual operations; since our Numba kernels are fused we time the entire kernel end to end. For Qwen, `resize` is the largest individual HF cost at 35%, but `patch layout` and `output assembly` together account for another 34%, while `rescale/normalize` contributes 9%. These are the sorts of operations dominated by memory movement that we can optimize with a scheduling DSL. Indeed, the DSL fuses sampling, pointwise operations, patchification, and the final output write into one compiled processing loop, reducing total profiled time by 16%.

LLaVA behaves very differently: HF Bilinear is dominated by input conversion and resizing, and the rest of the operations almost do not matter at all. Because LLaVA v1 and v3 have nearly identical W4 runtimes despite v3 eliminating intermediate buffers (Result 4), this grouped DSL cost is primarily attributed to the expensive AnyRes downsampling itself. Thus, scheduling removes meaningful layout and assembly overhead for Qwen, but cannot overcome LLaVA’s `resize`-dominated compute floor, causing DSL v3 to be slower than HF Bilinear.

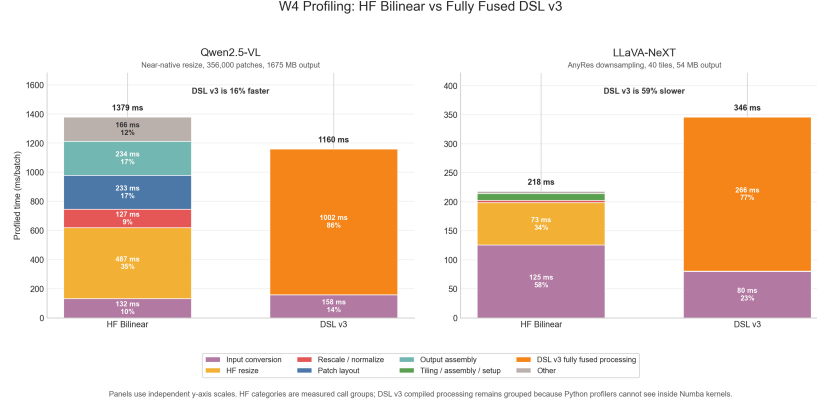


Figure 5: **Profiling Qwen and LLaVA.** Comparing HF Bilinear to DSL v3 for Qwen and LLaVA models. Profiling reports the amount of time (ms/full batch) spent in each function.

3.8 Measurement of Success

The results provide qualified support for the hypothesis that VLM image pre-processing benefits from algorithm–schedule separation.

1. **The DSL improves on open-source implementations.** All DSL schedules reproduce the algorithm-matched Hugging Face bilinear outputs within the stated floating-point tolerance. The strongest improvement is peak memory usage. In single-threaded execution, DSL v3 reaches $1.00\times$ the output size on Qwen W2 and W3, $1.08\times$ on Qwen W4, and $1.00\times$ on every LLaVA workload. In contrast, the Hugging Face implementations generally require approximately $1.7\text{--}4.1\times$ the output size. Runtime improvement is workload-dependent: the DSL is competitive with or faster than Hugging Face on several eight-thread Qwen and LLaVA workloads, but does not win universally.
2. **Schedule choice changes performance.** The Qwen schedules produce equivalent outputs but explore different memory and runtime behavior. On the eight-thread W4 workload, peak memory falls from $1.88\times$ for v1 to $1.13\times$ for v3 as the schedule moves from staged execution to full fusion and direct output writes. The batch-parallel Qwen schedule also scales from 994 ms per image with one thread to 144 ms with eight threads on W4, allowing it to overtake the algorithm-matched Hugging Face baseline. These results show that storage and parallelism decisions are meaningful optimization axes.
3. **The abstraction generalizes, but its performance benefit is model-dependent.** The same DSL primitives express both Qwen and LLaVA-NeXT despite their different output layouts and dynamic control logic.

LLaVA validates the expressiveness of the abstraction and receives the same strong memory benefit. However, its v1, v2, and v3 runtimes are nearly identical, and the DSL is slower than Hugging Face on the large W4 workload. Profiling explains this result: LLaVA is dominated by interpolation compute rather than removable intermediate-buffer overhead.

We conclude that VLM preprocessing is a useful application for algorithm-schedule separation when the pipeline contains substantial data movement, layout conversion, or output assembly. Scheduling is especially effective for reducing peak memory and can improve runtime when those costs are significant. It cannot replace a highly optimized interpolation kernel when resizing is the dominant computation.

4 Conclusion and Future Work

Existing VLM preprocessing implementations leave significant memory efficiency on the table. Our results show that a Halide-style separation of algorithm and schedule can successfully optimize these pipelines, with the clearest benefit coming from fusion, direct output writes, and reduced peak memory. Future work could expand the current template backend into a more general DSL that compiles scheduling primitives into generated kernels, add finer-grained parallelism, and develop optimized bilinear and bicubic interpolation kernels to close the remaining runtime gap with Hugging Face on resize-dominated workloads such as LLaVA. As VLM preprocessing grows more complex and consumes a larger fraction of end-to-end execution time, system designers must remember to optimize the CPU input pipeline alongside the model running on the GPU.

5 Team Responsibilities

Solo project. All design, implementation (IR, schedule language, op/coord registries, codegen, Numba kernels), benchmark and memory harness, correctness tests, experiments, and writing were performed by Julien Darve.

References

- [1] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, “Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines,” in *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI ’13, Seattle, Washington, USA: Association for Computing Machinery, 2013, pp. 519–530, ISBN: 9781450320146. DOI: 10.1145/2491956.2462176. [Online]. Available: <https://doi.org/10.1145/2491956.2462176>.

Table 1: Qwen2.5-VL multi-threaded median runtime in ms/image using 8 threads.

Workload	HF Legacy	HF Fast	HF Bilinear	DSL v1	DSL v2	DSL v3
W2	55.704	18.231	17.998	15.914	15.726	17.280
W3	21.571	5.613	5.514	7.756	7.333	8.525
W4	691.746	164.099	163.135	166.091	150.702	143.880

Table 2: Qwen2.5-VL peak RSS relative to output memory using 8 threads.

Workload	HF Legacy	HF Fast	HF Bilinear	DSL v1	DSL v2	DSL v3
W2	2.10×	3.89×	3.75×	1.40×	1.10×	1.10×
W3	1.99×	2.28×	2.32×	1.07×	1.00×	1.00×
W4	2.02×	3.86×	3.84×	1.88×	1.83×	1.13×

- [2] B. Li et al., “Llava-onevision: Easy visual task transfer,” *arXiv preprint arXiv:2408.03326*, 2024.
- [3] W. Wang et al., *Internvl3.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency*, 2025. arXiv: 2508.18265 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2508.18265>.
- [4] S. Bai et al., *Qwen2.5-vl technical report*, 2025. arXiv: 2502.13923 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2502.13923>.
- [5] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A llvm-based python jit compiler,” in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, ser. LLVM ’15, Austin, Texas: Association for Computing Machinery, 2015, ISBN: 9781450340052. DOI: 10.1145/2833157.2833162. [Online]. Available: <https://doi.org/10.1145/2833157.2833162>.
- [6] T. Wolf et al., *Huggingface’s transformers: State-of-the-art natural language processing*, 2020. arXiv: 1910.03771 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/1910.03771>.

6 Appendix

Here, we show the full result values. The results are all permutations of: {Qwen × LLaVA} × {8 × 4 × 1 threads} × {Runtime × Memory}. Within each chart, we display {W2 × W2 × W4} × {HF Legacy × HF Fast × HF Bilinear × DSL v1 × DSL v2 × DSL v3}. We have a total of 12 × 18 = 216 numbers!

Table 3: Qwen2.5-VL median runtime in ms/image using 4 threads.

Workload	HF Legacy	HF Fast	HF Bilinear	DSL v1	DSL v2	DSL v3
W2	63.949	27.901	27.600	28.766	26.693	31.301
W3	23.438	7.797	7.671	12.324	11.618	13.633
W4	725.749	244.217	241.679	289.493	256.362	265.060

Table 4: Qwen2.5-VL peak RSS relative to output memory using 4 threads.

Workload	HF Legacy	HF Fast	HF Bilinear	DSL v1	DSL v2	DSL v3
W2	2.08×	3.88×	3.76×	1.14×	1.00×	1.00×
W3	1.91×	2.29×	2.30×	1.00×	1.00×	1.00×
W4	2.00×	3.84×	3.84×	1.50×	1.46×	1.08×

Table 5: Qwen2.5-VL single-threaded median runtime in ms/image.

Workload	HF Legacy	HF Fast	HF Bilinear	DSL v1	DSL v2	DSL v3
W2	64.076	74.576	74.271	108.374	102.418	121.225
W3	23.522	19.815	19.871	43.902	41.215	48.748
W4	729.148	640.630	637.793	1070.222	953.593	994.205

Table 6: Qwen2.5-VL single-threaded peak RSS relative to output memory.

Workload	HF Legacy	HF Fast	HF Bilinear	DSL v1	DSL v2	DSL v3
W2	2.08×	3.88×	3.76×	1.00×	1.00×	1.00×
W3	1.91×	2.30×	2.31×	1.00×	1.00×	1.00×
W4	2.02×	3.84×	3.84×	1.20×	1.20×	1.08×

Table 7: LLaVA-NeXT multi-threaded median runtime in ms/image using 8 threads.

Workload	HF Legacy	HF Fast	HF Bilinear	DSL v1	DSL v2	DSL v3
W2	37.085	6.603	6.356	6.596	6.536	6.612
W3	23.631	4.314	4.195	3.686	3.602	3.646
W4	156.707	28.836	26.220	44.341	44.293	44.528

Table 8: LLaVA-NeXT peak RSS relative to output memory using 8 threads.

Workload	HF Legacy	HF Fast	HF Bilinear	DSL v1	DSL v2	DSL v3
W2	3.44×	4.05×	3.98×	1.06×	1.00×	1.00×
W3	1.84×	1.97×	2.49×	1.00×	1.00×	1.00×
W4	3.89×	4.26×	3.92×	2.44×	2.44×	2.44×

Table 9: LLaVA-NeXT median runtime in ms/image using 4 threads.

Workload	HF Legacy	HF Fast	HF Bilinear	DSL v1	DSL v2	DSL v3
W2	39.626	7.826	7.362	11.908	11.805	11.896
W3	23.132	5.408	5.214	6.640	6.531	6.555
W4	157.478	33.903	29.540	70.745	70.626	70.488

Table 10: LLaVA-NeXT peak RSS relative to output memory using 4 threads.

Workload	HF Legacy	HF Fast	HF Bilinear	DSL v1	DSL v2	DSL v3
W2	3.44×	4.04×	2.66×	1.00×	1.00×	1.00×
W3	2.59×	2.74×	2.37×	1.00×	1.00×	1.00×
W4	2.10×	2.49×	1.85×	1.00×	1.00×	1.00×

Table 11: LLaVA-NeXT single-threaded median runtime in ms/image.

Workload	HF Legacy	HF Fast	HF Bilinear	DSL v1	DSL v2	DSL v3
W2	37.941	15.019	14.663	44.001	43.165	43.971
W3	23.254	12.006	11.946	22.405	21.771	22.033
W4	157.016	36.490	33.912	252.443	251.881	251.332

Table 12: LLaVA-NeXT single-threaded peak RSS relative to output memory.

Workload	HF Legacy	HF Fast	HF Bilinear	DSL v1	DSL v2	DSL v3
W2	3.44×	4.05×	3.98×	1.00×	1.00×	1.00×
W3	1.66×	1.77×	1.75×	1.00×	1.00×	1.00×
W4	3.39×	3.07×	1.83×	1.00×	1.00×	1.00×